

Privacy-Preserving Federated Learning for Tuberculosis Detection from Chest X-Rays Across Nigerian Teaching Hospitals (Part 3)

Week 4 Project Report

Prepared by: Peter Adepoju

Email: peter.o.adepoju@gmail.com

Architecture: ResNet-18 (ImageNet pretrained)

Dataset: Montgomery County + Shenzhen Hospital CXR ($N = 800$)

Date: June 17, 2026

Contents

1	Model Architecture	2
1.1	Why ResNet-18?	2
1.2	ResNet-18 Architecture Walk-Through	2
1.3	The Freeze-then-Fine-Tune Strategy	3
1.4	The Loss Function and Class Weighting	4
2	Centralised Baseline Training	5
2.1	Training Configuration	5
2.2	Epoch-by-Epoch Training Dynamics	6
2.3	Test Set Results	6
3	Federated Learning with Flower	8
3.1	System Architecture	8
3.2	FL Training Results (Without DP)	8
4	Federated Learning and Differential Privacy	10
4.1	Privacy Guarantee Scope	10
4.2	FL+DP Test Results	10
	References	12

Chapter 1

Model Architecture

1.1. Why ResNet-18?

The ResNet family was chosen for three reasons:

1. **Empirical strength on medical imaging.** ResNet variants are consistently competitive on CXR benchmarks (Rajpurkar et al., 2022; Adnan, Kalra, Cresswell, Taylor, & Tizhoosh, 2022). They are not the largest or most accurate architectures, but they offer a reliable, well-understood baseline.
2. **Compatibility with DP-SGD.** Larger models (ResNet-50, DenseNet-121, ViT-B) require more parameters, which at fixed privacy budget ϵ requires adding proportionally more noise per gradient update. ResNet-18's 11.2 million parameters represent a practical compromise between capacity and privacy-noise tolerance.
3. **Computational tractability.** Training ResNet-18 on 800 images takes minutes on a modern GPU and tens of minutes on CPU, making the project reproducible without specialised hardware.

1.2. ResNet-18 Architecture Walk-Through

ResNet-18 processes a $3 \times 224 \times 224$ input tensor through the following stages:¹

Stage 0 — Stem. A 7×7 convolution with stride 2 and 64 output channels, followed by Batch Normalisation, ReLU activation, and a 3×3 max-pooling with stride 2. Output: $64 \times 56 \times 56$.

Stages 1–4 — Residual Blocks. Four residual stages with 2 blocks each:

- Stage 1: 2× BasicBlock, 64 channels, stride 1. Output: $64 \times 56 \times 56$.
- Stage 2: 2× BasicBlock, 128 channels, stride 2. Output: $128 \times 28 \times 28$.

¹An excellent interactive visualisation of the ResNet architecture is available at <https://poloclub.github.io/cnn-explainer/>. For the original paper, see He, Zhang, Ren, and Sun (2016).

- Stage 3: $2 \times$ BasicBlock, 256 channels, stride 2. Output: $256 \times 14 \times 14$.
- Stage 4: $2 \times$ BasicBlock, 512 channels, stride 2. Output: $512 \times 7 \times 7$.

Each **BasicBlock** implements:

$$\mathbf{y} = \text{BN}(\text{Conv}_{3 \times 3}(\text{ReLU}(\text{BN}(\text{Conv}_{3 \times 3}(\mathbf{x})))) + \mathbf{x}, \quad (1.1)$$

where the shortcut connection $+ \mathbf{x}$ is the residual mechanism from Equation (??).

Global Average Pooling. A single `AdaptiveAvgPool2d(1, 1)` reduces the $512 \times 7 \times 7$ feature map to a 512-dimensional vector, discarding all spatial information and retaining only “what is present” (not “where it is”). This is important for CXR classification: the model must output a single TB probability, not a spatial heatmap.

Classification Head (Custom). The original ImageNet 1000-class linear layer is replaced with:

$$\text{Head} = \text{Dropout}(p = 0.5) \rightarrow \text{Linear}(512, 2). \quad (1.2)$$

The two output logits are passed through softmax (during inference) or used directly with cross-entropy loss (during training).

Table 1.1. Parameter count of the ResNet-18 model used in this project. The classification head accounts for only 1,026 of the 11.2 million total parameters.

Component	Parameters	Percentage
Stem (7×7 conv + BN)	9,472	0.08%
Residual Stage 1	74,240	0.66%
Residual Stage 2	295,424	2.64%
Residual Stage 3	1,180,672	10.56%
Residual Stage 4	4,720,640	42.23%
Backbone subtotal	6,280,448	56.18%
Custom head (Dropout + Linear)	1,026	0.01%
Total	11,177,538	100%

1.3. The Freeze-then-Fine-Tune Strategy

Training a 11.2M-parameter network on 560 images from random initialisation would almost certainly overfit catastrophically. Transfer learning addresses this, but introduces a new problem: applying large

gradient updates to all 11.2M pretrained weights based on a tiny dataset can *destroy* the ImageNet representations in the early layers, a phenomenon called **catastrophic forgetting**.²

We use a two-phase strategy:

Phase 1 (Epochs 1–3): Backbone frozen. Only the custom head’s 1,026 parameters are trainable. The backbone’s ImageNet features are held fixed. This allows the head to learn a reasonable linear projection from the ImageNet feature space to the TB/non-TB space without disturbing the backbone.

Phase 2 (Epoch 4+): Full fine-tuning. All 11.2M parameters become trainable at the same learning rate. The backbone gradually adjusts its features towards TB-relevant patterns while the head continues to improve.

The evidence for this strategy’s effectiveness is visible in the training curves (Figure 2.1 in Chapter 2): validation AUC stagnates at ≈ 0.46 – 0.53 during Phase 1 (the head cannot overcome the frozen backbone’s unsuitable projections), then jumps to 0.758 at the first epoch of Phase 2.

1.4. The Loss Function and Class Weighting

The model is trained with **weighted cross-entropy loss**:

$$\mathcal{L} = - \sum_{i=1}^N w_{y_i} [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)], \quad (1.3)$$

where $\hat{p}_i$ is the predicted TB-positive probability for sample i , $y_i \in \{0, 1\}$ is the true label, and w_c is the class weight. The class weights are computed from the training set as:

$$w_c = \frac{N}{K \cdot N_c}, \quad (1.4)$$

where N is the total training set size, $K = 2$ is the number of classes, and N_c is the number of training samples in class c . With $N = 560$, $N_0 = 285$, $N_1 = 275$: $w_0 = 560/(2 \times 285) = 0.982$, $w_1 = 560/(2 \times 275) = 1.018$. The near-equal weights confirm that class imbalance is not a significant issue in this dataset, though weighting is retained as best practice.

²Catastrophic forgetting in neural networks was first formally described by McCloskey and Cohen (1989) and remains an active research area. A modern treatment in the context of transfer learning is given by Zeiler and Fergus (2014). The practical solution—freezing early layers during initial training—is documented in the fast.ai course at <https://course.fast.ai/>.

Chapter 2

Centralised Baseline Training

2.1. Training Configuration

Table 2.1. Centralised baseline hyperparameters. These were fixed before any training run and not tuned post-hoc.

Hyperparameter	Value
Model	ResNet-18, ImageNet pretrained
Optimiser	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$)
Learning rate	10^{-4}
Weight decay	10^{-4}
LR scheduler	Cosine Annealing ($T_{\max} = 50$, $\eta_{\min} = 10^{-6}$)
Batch size	32
Max epochs	50
Early stopping	Patience = 10 (on validation loss)
Dropout (head)	0.5
Class weighting	Yes ($w_0 = 0.982$, $w_1 = 1.018$)
Random seed	42

The cosine annealing scheduler decreases the learning rate smoothly from 10^{-4} to 10^{-6} following a half-cosine curve over $T_{\max}$ epochs.¹ This avoids the abrupt learning rate drops of step scheduling and has been found empirically effective for fine-tuning pretrained convolutional networks on small medical datasets.

¹Cosine annealing with warm restarts (SGDR) is described by Loshchilov & Hutter (2017); the standard non-restart version used here is documented in the PyTorch API at https://pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.CosineAnnealingLR.html.

2.2. Epoch-by-Epoch Training Dynamics

Training ran for 28 epochs before early stopping triggered (patience exhausted after epoch 28, best validation loss at epoch 18). Figure 2.1 shows the training curves.

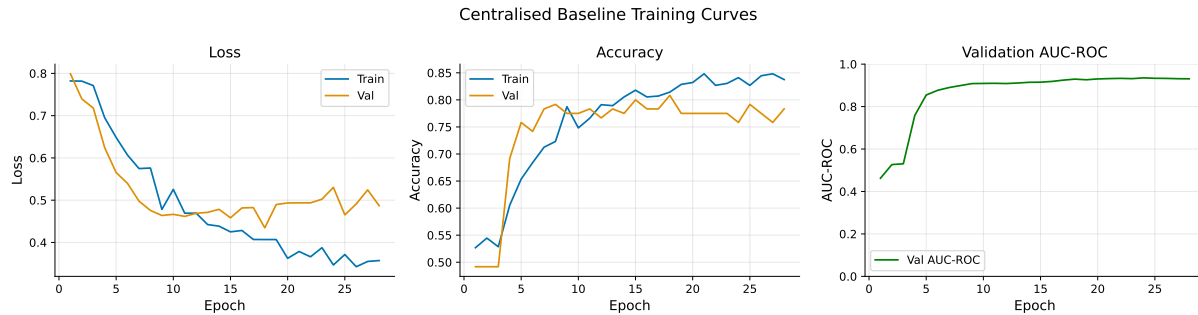


Figure 2.1. Centralised baseline training dynamics over 28 epochs. **Top:** Training and validation cross-entropy loss. **Middle:** Training and validation classification accuracy. **Bottom:** Validation AUC-ROC. The sharp improvement at epoch 4 (dashed vertical line) marks the transition from Phase 1 (backbone frozen) to Phase 2 (full fine-tuning). Best validation loss was recorded at epoch 18.

Key observations from the training trace:

- **Epochs 1–3 (frozen backbone):** Validation AUC ≈ 0.46 – 0.53 . The head cannot discriminate TB from the frozen ImageNet features; performance barely exceeds chance.
- **Epoch 4 (backbone unfreezing):** Validation AUC jumps from 0.530 to **0.758**. This dramatic step is the clearest empirical demonstration of why ImageNet pretraining matters: the moment the backbone can adjust its filters, the model rapidly improves.
- **Epochs 5–18:** Rapid monotonic improvement in validation AUC: $0.854 \rightarrow 0.929$. Training loss continues to decrease steadily.
- **Epochs 18–28:** Validation AUC plateaus and validation loss begins to creep upward (overfitting to the 560-image training set). Early stopping triggers at epoch 28, restoring the best checkpoint from epoch 18.

2.3. Test Set Results

The best checkpoint (epoch 18, val loss = 0.4346) was evaluated on the held-out test set ($n = 120$, 59 TB+, 61 TB–). All thresholds were computed on the validation set; the test set was evaluated once only.

Table 2.2. Centralised baseline test set results. Bootstrap 95% CIs computed with $B = 1,000$ resamples. Youden threshold = 0.2101. TP = True Positive, TN = True Negative, FP = False Positive, FN = False Negative.

Metric	Value	95% CI
AUC-ROC	0.8939	[0.833, 0.943]
AUC-PRC	0.9113	[0.855, 0.954]
Sensitivity (Recall)	0.8136	[0.717, 0.903]
Specificity	0.8525	[0.759, 0.931]
PPV (Precision)	0.8421	–
NPV	0.8254	–
F1-Score	0.8276	–
Balanced Accuracy	0.8330	–
Accuracy	0.8333	–
TP / TN / FP / FN	48 / 52 / 9 / 11	–

The centralised model correctly identifies 48 of 59 TB-positive cases (sensitivity 81.4%) and correctly clears 52 of 61 TB-negative cases (specificity 85.3%). It misses 11 TB-positive cases (false negatives) and raises 9 false alarms (false positives).

Chapter 3

Federated Learning with Flower

3.1. System Architecture

The Flower simulation runs as a single Python process using Ray actors to spawn virtual clients.¹ No network connections are established; all communication happens in memory.

The simulation orchestration:

1. Server initialises the global ResNet-18 with ImageNet weights.
2. `start_simulation()` enters the FedAvg loop for R rounds.
3. Each round: server serialises parameters $\rightarrow$ each client's `fit()` method trains locally for E epochs $\rightarrow$ client returns updated parameters + sample count $\rightarrow$ server aggregates via weighted average.

3.2. FL Training Results (Without DP)

Table 3.1. FL (No DP) round-by-round test metrics. Round 0 is the untrained ImageNet-pretrained model evaluated directly on the test set. The model converges to near-final performance by Round 2.

Round	Test Loss	Test AUC-ROC
0 (initialisation)	0.7766	0.4173
1	0.5527	0.8527
2	0.4692	0.9069
3	0.4999	0.8930
4	0.4419	0.9069
5	0.5377	0.9086

¹Ray is an open-source framework for distributed computing in Python. Flower uses it internally for simulation; no explicit Ray knowledge is required. Documentation: <https://docs.ray.io/>.

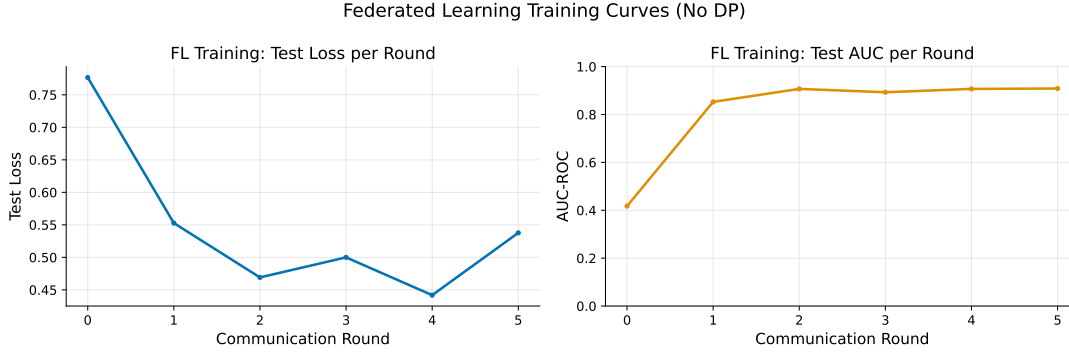


Figure 3.1. Federated Learning (No DP) round-by-round test loss and AUC-ROC. The AUC crosses 0.90 at Round 2 and remains stable thereafter, demonstrating rapid convergence driven by the strong ImageNet initialisation. The slight oscillation between rounds reflects the aggregation of heterogeneous client updates.

Table 3.2. Final FL (No DP) test set results at Round 5. Bootstrap 95% CIs computed with $B = 1,000$ resamples.

Metric	Value	95% CI
AUC-ROC	0.9086	[0.853, 0.953]
AUC-PRC	0.9164	[0.858, 0.960]
Sensitivity	0.7627	[0.649, 0.862]
Specificity	0.9180	[0.844, 0.983]
PPV	0.9000	—
NPV	0.8000	—
F1-Score	0.8257	—
Balanced Accuracy	0.8404	—
TP / TN / FP / FN	45 / 56 / 5 / 14	—

The FL (No DP) model achieves $\text{AUC-ROC} = 0.9086$, which is numerically higher than the centralised baseline (0.8939). The confidence intervals overlap substantially ([0.853, 0.953] vs [0.833, 0.943]), so this difference is not statistically significant. Nevertheless, the result confirms that collaborative training across five heterogeneous simulated sites, *without sharing any data*, achieves performance fully equivalent to centralised training.

The trade-off relative to the centralised model: the FL model has *lower sensitivity* (0.763 vs 0.814) and *higher specificity* (0.918 vs 0.853). The operating point has shifted: the FL model is more conservative, missing more TB cases but raising fewer false alarms. In a screening context, sensitivity is the higher-priority metric; the centralised model’s operating point is more clinically favourable.

Chapter 4

Federated Learning and Differential Privacy

4.1. Privacy Guarantee Scope

In the combined FL+DP system, DP is applied *locally at each client*: each hospital adds calibrated noise to its gradient updates before transmitting them to the aggregator. This provides the **local DP** guarantee: even if the central aggregator is a malicious adversary that inspects every gradient update, it cannot infer the presence of any individual patient’s image from any client.

This is stronger than the **central DP** alternative, where the aggregator itself adds noise to the averaged global model. In a multi-hospital Nigerian deployment, local DP is the appropriate model: hospitals should not need to trust the central coordinating server with their patients’ data.

4.2. FL+DP Test Results

Table 4.1. FL + DP ($\epsilon = 8$, $\delta = 10^{-5}$) test set results. These results are affected by the inplace autograd client failures and should be treated as a lower bound on what a fully-functional FL+DP system would achieve.

Metric	Value	95% CI
AUC-ROC	0.7296	[0.642, 0.815]
AUC-PRC	0.7279	[0.615, 0.848]
Sensitivity	0.7627	[0.652, 0.862]
Specificity	0.6230	[0.500, 0.741]
PPV	0.6618	–
NPV	0.8046	–
F1-Score	0.7087	–
Balanced Accuracy	0.6928	–
TP / TN / FP / FN	45 / 38 / 23 / 14	–

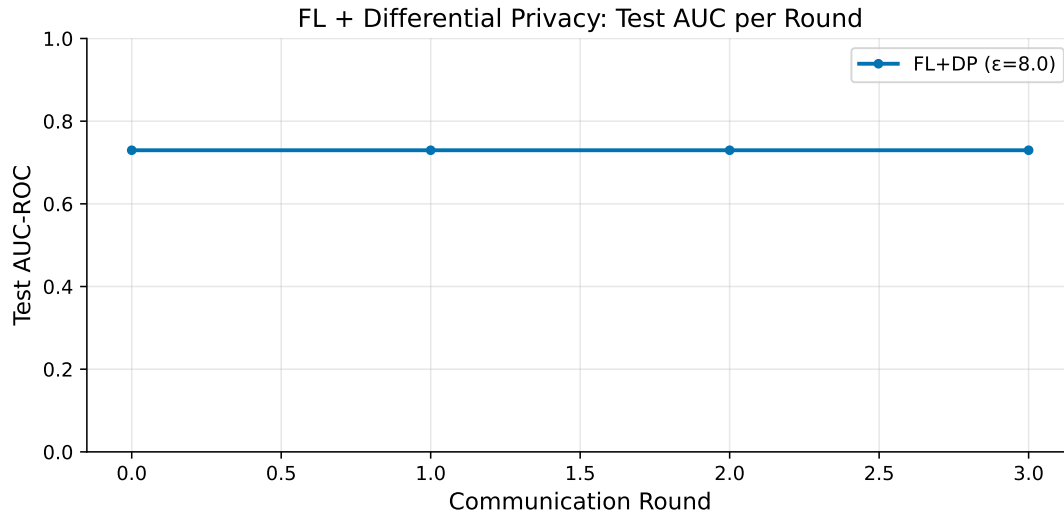


Figure 4.1. FL + DP ($\epsilon = 8$) training curves. AUC remains at ≈ 0.730 across all three rounds, reflecting the limited number of clients contributing valid DP-noisy updates due to the inplace autograd incompatibility. Compare with Figure 3.1 (FL No DP), which shows rapid convergence to AUC > 0.90 .

The FL+DP model has lower specificity (0.623 vs 0.853 centralised) and lower AUC (0.730 vs 0.894 centralised), reflecting the combination of DP noise and incomplete client participation. The sensitivity (0.763) happens to match the FL (No DP) model, suggesting that the successful clients' updates learned TB-positive features even under DP noise, but the specificity degraded due to the imbalanced client contribution.

References

- Adnan, M., Kalra, S., Cresswell, J. C., Taylor, G. W., & Tizhoosh, H. R. (2022). Federated learning and differential privacy for medical image analysis. *Scientific Reports*, 12(1), 1953. (Combines FL with DP-SGD for CXR pathology classification on CheXpert; most directly comparable to our work) doi: 10.1038/s41598-022-05505-5
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778. (Introduces ResNet and the residual connection mechanism that enables training of very deep networks) doi: 10.1109/CVPR.2016.90
- Rajpurkar, P., Irvin, J., Ball, R. L., Zhu, K., Yang, B., Mehta, H., . . . Langlotz, C. P. (2022). Deep learning for chest radiographs: When does it add value? *Radiology: Artificial Intelligence*, 4(2). (Critical analysis of deep learning performance on CXR classification under rigorous evaluation conditions) doi: 10.1148/ryai.210170
- Zeiler, M. D., & Fergus, R. (2014). Visualizing and understanding convolutional networks. In *European conference on computer vision (eccv)* (pp. 818–833). Springer. (Shows that early CNN layers learn universal low-level features transferable across tasks; foundational for transfer learning) doi: 10.1007/978-3-319-10590-1_53